HW1 Gene Expression

Dylan Renard

Professor Brandon Higgs

Gene Expression and Data Visualization

Johns Hopkins University

September 30th, 2025

HW1:

For this homework, we will be working with a study from Gene Expression Omnibus (GEO) with the accession GDS2880. This is an Affymetrix microarray experiment (HGU133A array). The data researchers were investigating patient matched normal and stage 1 or stage 2 clear cell renal cell carcinoma (cRCC) tumors to provide insight into the molecular pathogenesis of cRCC. We will be conducting outlier analysis using various methods to identify aberrant samples, followed by missing value imputation to assess the accuracy of two different algorithms.

▼ Code

```
# Library snippet
# Load libraries
library(GEOquery)
library(Biobase)
library(stringr)
library(dplyr)
library(tibble)
library(pheatmap)
library(dendextend)
library(ggrepel)
library(ggplot2)
library(impute)
library(reshape2)
library(AnnotationDbi)
library(hgu133a.db)
library(pcaMethods)
```

1)

Download and load the renal cell carcinoma data file into R. Make sure that the row names are in the correct location (Affymetrix fragment names). Look at the dimensions and verify that you have 22 arrays

and 22,283 probesets.

▼ Code

```
# Download dataset (will cache locally after first time)
gds <- getGEO("GDS2880")

# Convert GEO GDS object to ExpressionSet (standard Bioconductor format)
eset <- GDS2eSet(gds, do.log2 = FALSE)

# Expression matrix
expr <- Biobase::exprs(eset)

# Phenotype (sample metadata)
pheno <- Biobase::pData(eset)

# Quick checks
dim(expr)        # dimensions: probes x samples
```

```
[1] 22283    20
```

▼ Code

```
head(rownames(expr))  # should show Affymetrix probe IDs
```

```
[1] "1007_s_at" "1053_at"   "117_at"    "121_at"    "1255_g_at" "1294_at"
```

▼ Code

```
head(colnames(expr))  # GSM sample IDs
```

```
[1] "GSM146778" "GSM146780" "GSM146782" "GSM146784" "GSM146786" "GSM146789"
```

2)

Label the header columns of your data frame maintaining the GSM ID, but adding the Normal/Tumor identity.

▼ Code

```
# 1) Inspect metadata columns
meta_cols <- colnames(pheno)

# 2) Fuse likely fields into a single searchable string per sample
cand_cols <- grep(
  paste(c("title","source","character","descr","disease","tissue"), collapse="|"),
  meta_cols, ignore.case = TRUE, value = TRUE
)
```

```
if (length(cand_cols) == 0) cand_cols <- meta_cols

meta_text <- apply(pheno[, cand_cols, drop=FALSE], 1, function(row) {
  paste(na.omit(as.character(row)), collapse = " | ")
})

# 3) Regex rules
re_normal <- "(^|[^a-z])normal([^a-z]|$)|adjacent normal|non[- ]?tumou?r"
re_tumor  <- "tumou?r|rcc|renal cell carcinoma|clear cell|stage\\s*[12]"

# 4) Classify
condition <- rep(NA_character_, length(meta_text))
condition[grepl(re_normal, meta_text, ignore.case = TRUE)] <- "Normal"
condition[grepl(re_tumor,  meta_text, ignore.case = TRUE) & is.na(condition)] <- "Tumor"

# 5) Report unknowns (if any)
unknown_idx <- which(is.na(condition))
if (length(unknown_idx)) {
  cat("\n[WARN] Unclassified samples — review these rows:\n")
  print(data.frame(
    GSM  = rownames(pheno)[unknown_idx],
    Meta = meta_text[unknown_idx],
    check.names = FALSE
  ), row.names = FALSE)
  # stop("Unclassified samples present; refine regex or set manual overrides.")
}

# 6) Summary & apply names
cat("\nCondition summary:\n"); print(table(condition, useNA = "ifany"))
```

```
Condition summary:

condition
Normal  Tumor
    10     10
```

▼ Code

```
gsm <- rownames(pheno)
stopifnot(length(gsm) == ncol(expr))
colnames(expr) <- paste0(gsm, "_", condition)

head(colnames(expr))
```

```
[1] "GSM146778_Normal" "GSM146780_Normal" "GSM146782_Normal" "GSM146784_Normal"
[5] "GSM146786_Normal" "GSM146789_Normal"
```

3) Identify any outlier samples using the following visual plots:

For all plots, make sure you label the points appropriately, title plots, and label axes. You will also need to provide a legend for the correlation plot. You can use the gplots for a color gradient, or just use the default colors.

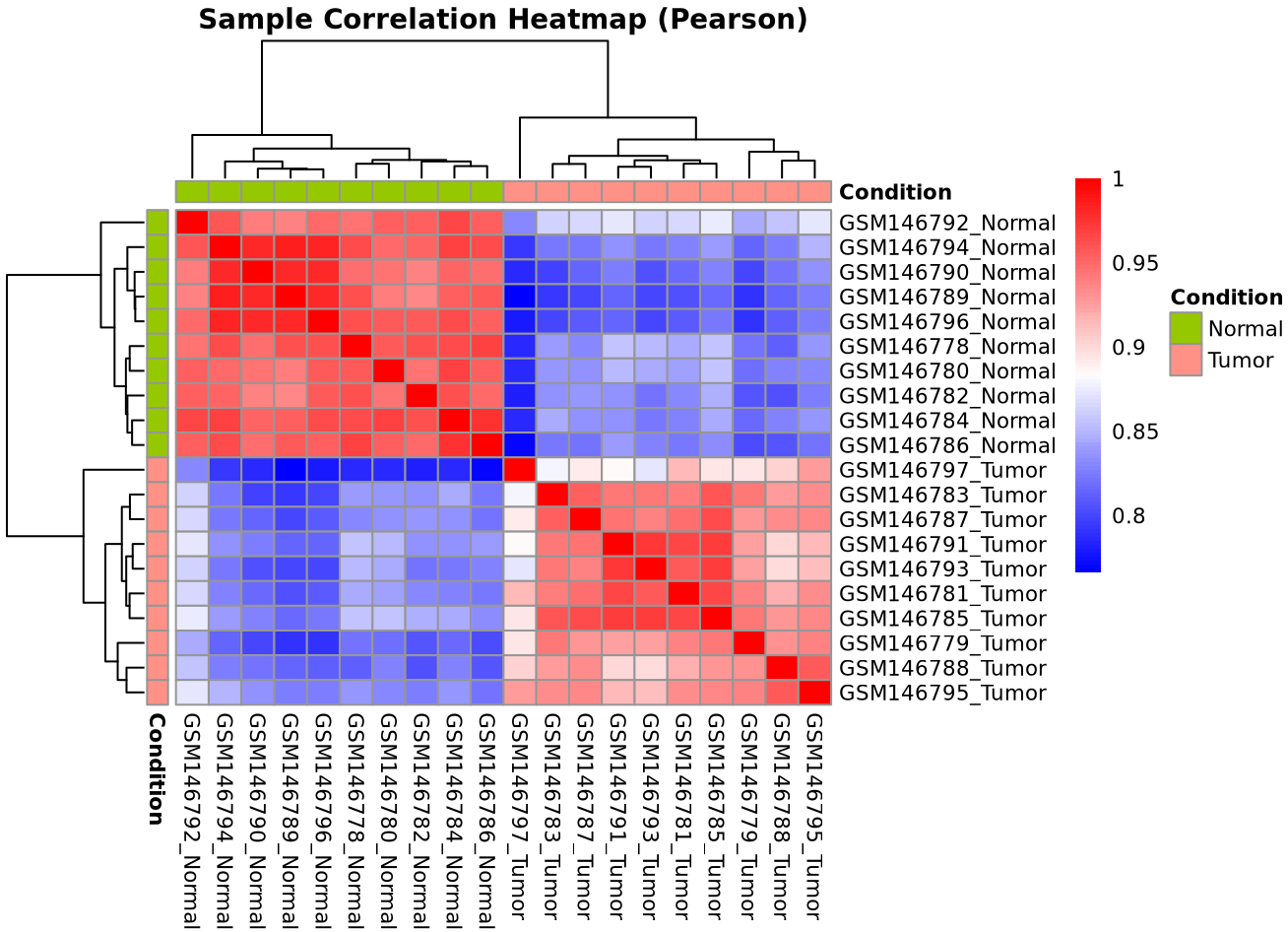
3a) Correlation plot (heat map)

▼ Code

```
# Compute correlation matrix
cors <- cor(expr, use = "pairwise.complete.obs", method = "pearson")

# Build annotation for sample groups
ann <- data.frame(Condition = condition)
rownames(ann) <- colnames(expr)

# Plot heatmap
pheatmap(cors,
         annotation_col = ann,
         annotation_row = ann,
         main = "Sample Correlation Heatmap (Pearson)",
         fontsize = 8,
         color = colorRampPalette(c("blue", "white", "red"))(100))
```



Interpreation of Correlation plot (heatmap)

The correlation heatmap shows the expected structure: Normals (blue) cluster tightly together and Tumors (red) cluster tightly together, while correlations between the two groups are much weaker. Within Normals, almost all arrays are strongly correlated, though GSM146790 and GSM146786 stand out with slightly weaker similarity to their peers. Within Tumors, most arrays are consistent, but GSM146793, GSM146783, and GSM146781 show reduced correlation inside the tumor block. In the cross-group comparisons (Normal vs Tumor), these same arrays — GSM146790, GSM146786, GSM146793, and GSM146781 — appear especially weak against the opposite group. Overall, the heatmap confirms strong separation between Normal and Tumor samples while highlighting a handful of arrays in each group that deviate modestly from the otherwise uniform block pattern.

3b) Hierarchical clustering dendogram

▼ Code

```
# Distance matrix from correlation (1 - Pearson r)
D  <- as.dist(1 - cors)
hc <- hclust(D, method = "average")

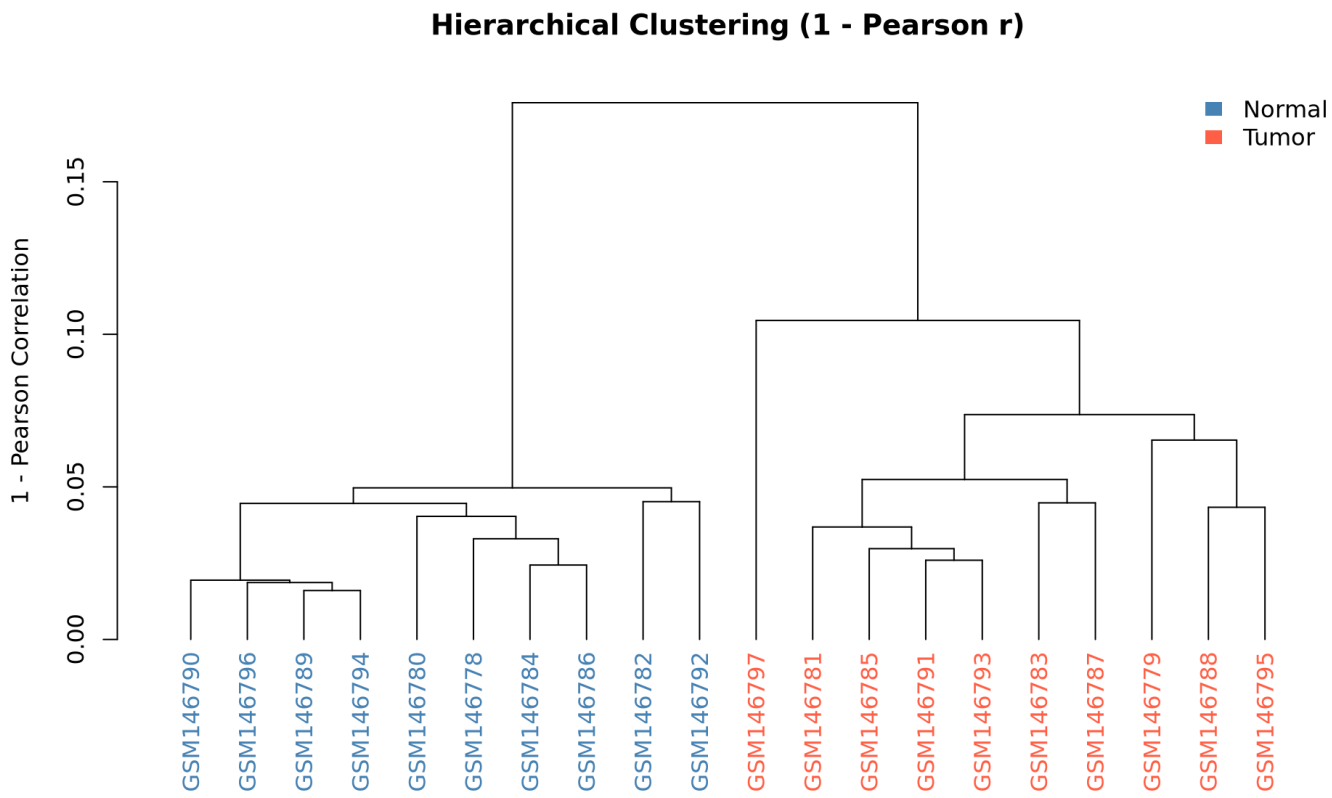
# Build dendrogram
dend <- as.dendrogram(hc)
```

```
# Short GSM-only labels (strip _Normal/_Tumor)
gsm_labels <- sub("_(Normal|Tumor)$", "", colnames(expr))

# IMPORTANT: set labels *on the dend* in the clustered order
labels(dend) <- gsm_labels[order.dendrogram(dend)]

# Color labels by group
group_cols <- ifelse(condition == "Normal", "steelblue", "tomato")
labels_colors(dend) <- group_cols[order.dendrogram(dend)]

# Plot
plot(dend,
     main = "Hierarchical Clustering (1 - Pearson r)",
     xlab = "",
     ylab = "1 - Pearson Correlation",
     xaxt ="n",
     cex = 0.8)   # shrink label text a bit
legend("topright",
       legend = c("Normal","Tumor"),
       fill = c("steelblue","tomato"),
       border = NA, bty = "n")
```



Hierarchical clustering dendrogram. Samples (GSM IDs) are ordered along the x-axis.

Interpretation of Dendogram

The hierarchical clustering dendrogram shows a clean split between Normal (blue) and Tumor (red) samples, confirming that the largest source of variation is biological condition. Within the Normal branch, most arrays cluster very tightly, but GSM146790 and GSM146786 are attached on longer twigs, suggesting they are less similar to their Normal peers. The Tumor branch also shows overall cohesion, though GSM146793, GSM146783, and GSM146781 sit farther from the main tumor cluster, reflecting weaker similarity. No sample crosses over into the opposite condition's branch, so group identity remains intact, but the dendrogram highlights a subset of arrays within each group that are less consistent.

3c) CV vs mean plot

▼ Code

```
# # Calculate per-sample mean and CV
samp_mean <- apply(expr, 2, function(x) mean(x, na.rm = TRUE))
samp_sd   <- apply(expr, 2, function(x) sd(x, na.rm = TRUE))
samp_cv   <- samp_sd / samp_mean

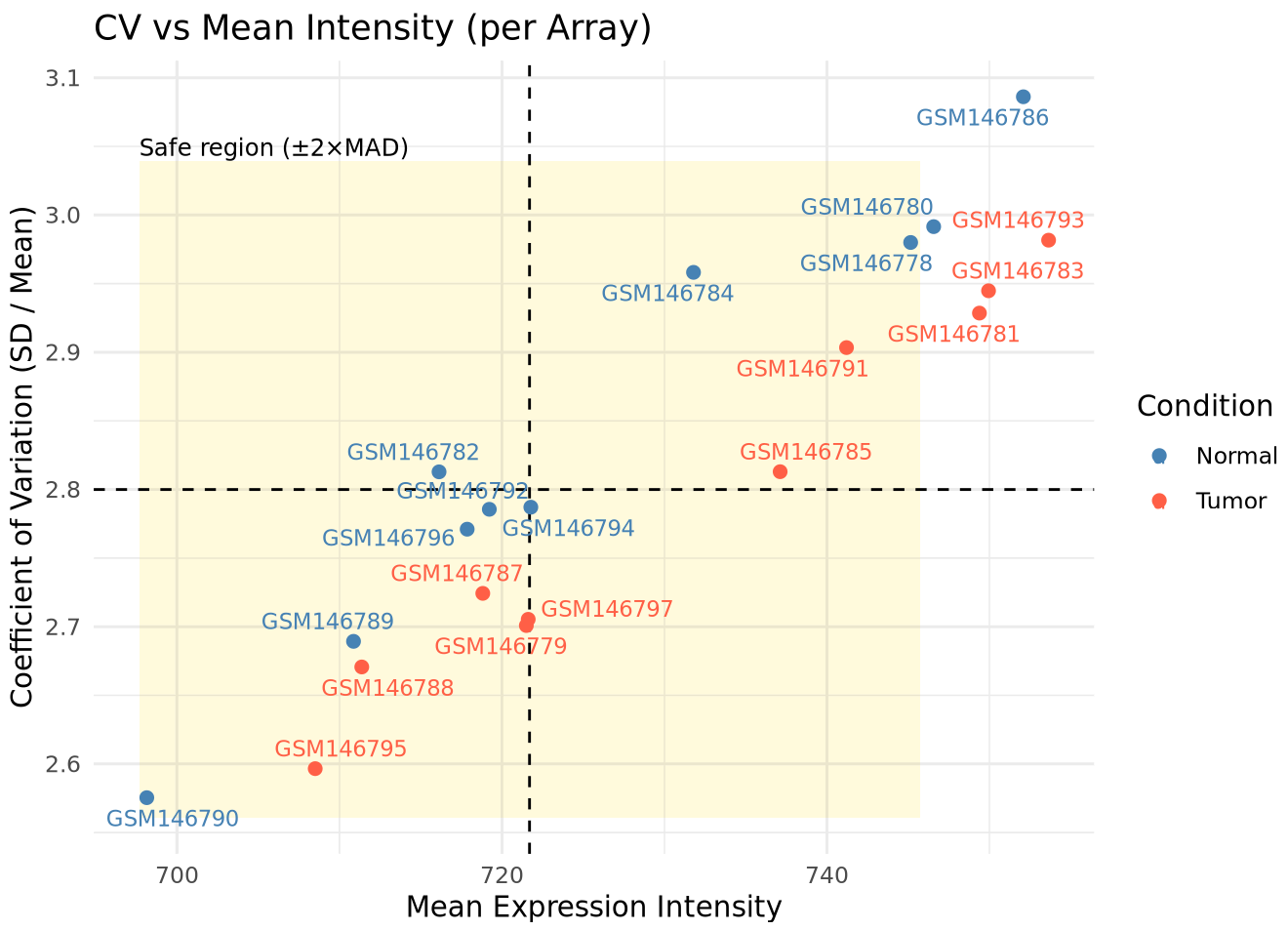
cvdf <- data.frame(
  GSMID     = sub("_(Normal|Tumor)$", "", colnames(expr)),  # short ID
  Condition = condition,
  Mean      = samp_mean,
  CV        = samp_cv
)

# Robust center & band
m_mean <- median(cvdf$Mean, na.rm = TRUE)
m_cv   <- median(cvdf$CV,   na.rm = TRUE)
b_mean <- 2 * mad(cvdf$Mean, constant = 1, na.rm = TRUE)  # width
b_cv   <- 2 * mad(cvdf$CV,   constant = 1, na.rm = TRUE)  # height

xmin <- m_mean - b_mean; xmax <- m_mean + b_mean
ymin <- m_cv   - b_cv;   ymax <- m_cv   + b_cv

ggplot(cvdf, aes(x = Mean, y = CV, color = Condition, label = GSMID)) +
  # SAFE REGION first (under points)
  annotate("rect", xmin = xmin, xmax = xmax, ymin = ymin, ymax = ymax,
           fill = "gold", alpha = 0.12, color = NA) +
  # Median reference lines
  geom_vline(xintercept = m_mean, linetype = "dashed") +
  geom_hline(yintercept = m_cv,   linetype = "dashed") +
  # Points & labels
  geom_point(size = 2) +
  ggrepel::geom_text_repel(size = 3, max.overlaps = 20) +
  # Caption label for the zone (optional)
  annotate("text", x = xmin, y = ymax, hjust = 0, vjust = -0.3,
           label = "Safe region (±2×MAD)", size = 3.2) +
  labs(title = "CV vs Mean Intensity (per Array)",
       x = "Mean Expression Intensity",
       y = "Coefficient of Variation (SD / Mean)") +
```

```
  scale_color_manual(values = c("Normal" = "steelblue", "Tumor" = "tomato")) +
  theme_minimal()
```



Interpretation of our CV vs Mean Intesity per Array Plot

The CV versus mean plot evaluates each array's overall **signal intensity** (x-axis) and **variability** (y-axis). To help identify well-behaved samples versus potential outliers, we overlaid a yellow "safe region" box defined using the **median absolute deviation (MAD)**.

MAD stands for *median absolute deviation*, which is a robust statistic for measuring spread:

$$MAD(x) = \text{median}\big(|x_i - \text{median}(x)|\big)$$

Because the median and MAD are not distorted by extreme values the way the mean and standard deviation can be, this approach provides a conservative way to flag arrays that differ too much from the majority. In this plot, the safe region is set as the **median ± 2 × MAD** for both mean intensity and CV. Arrays inside the box behave like the bulk of the data, while arrays outside are flagged as potential outliers.

Most arrays, both **Normals (blue)** and **Tumors (red)**, fall inside the safe region, showing balanced intensity and variability. Two Normals deviate strongly: **GSM146790** (low mean and low CV) and **GSM146786** (high mean and high CV). Several Tumors—**GSM146793, GSM146783, and GSM146781**—also lie above the upper-right boundary, while **GSM146780** and **GSM146778** sit directly on the border.

This analysis identifies **two Normal arrays (GSM146790 and GSM146786)** as strong technical outliers. A subset of Tumor arrays also deviate from the safe range; their higher signal and variability may reflect true biological heterogeneity in clear cell renal cell carcinoma, though they should be noted for caution in downstream analysis.

3d) Average correlation plot

▼ Code

```
# Compute average Pearson correlation per sample
cors <- cor(expr, use = "pairwise.complete.obs", method = "pearson")
avg_cor <- sapply(seq_len(ncol(cors)), function(i) mean(cors[i, -i], na.rm = TRUE))
names(avg_cor) <- sub("_(Normal|Tumor)$", "", colnames(expr))

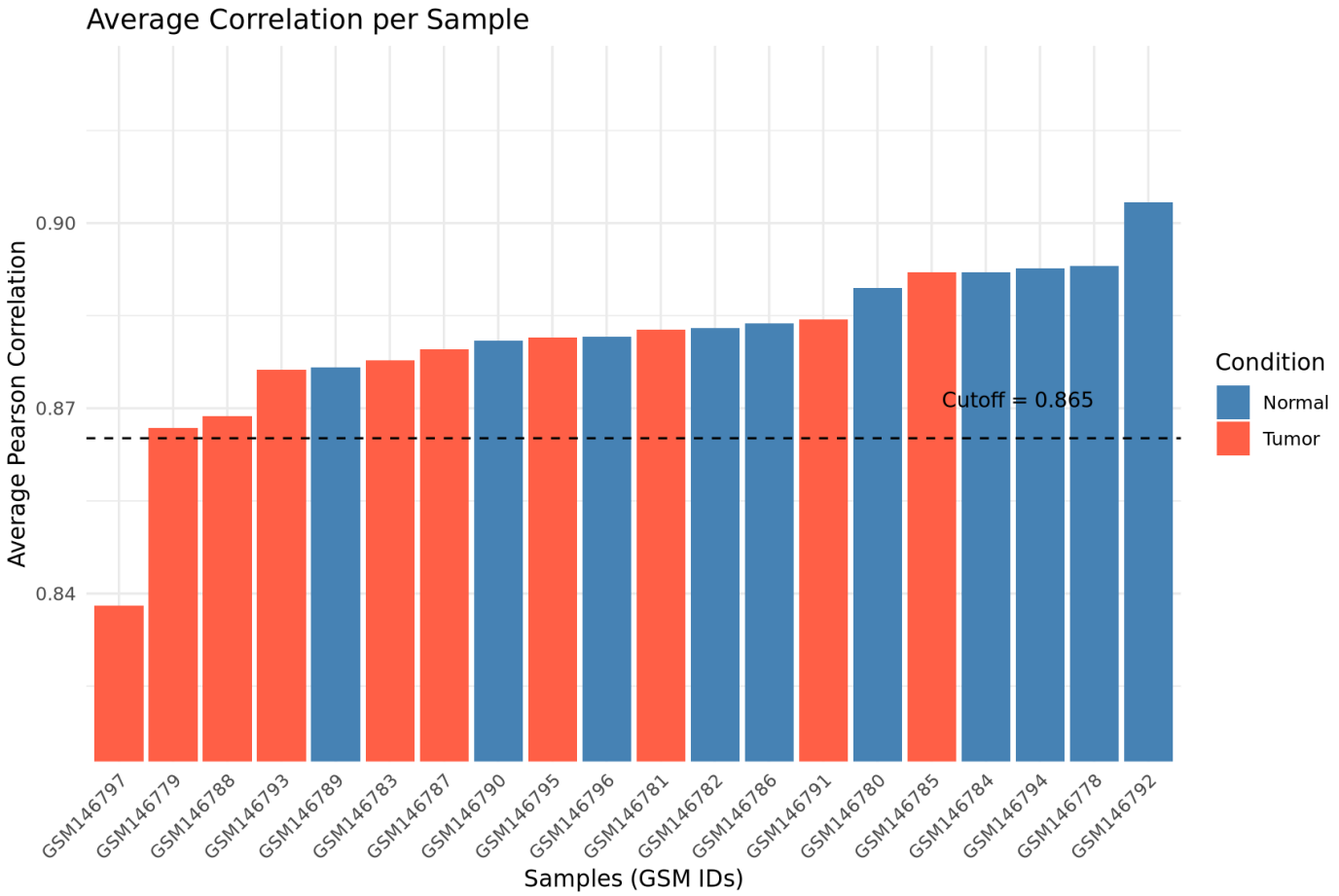
# Condition map
cond_map <- setNames(condition, sub("_(Normal|Tumor)$", "", colnames(expr)))

acdf <- data.frame(
  GSMID = names(avg_cor),
  Condition = unname(cond_map[names(avg_cor)]),
  AvgCor = as.numeric(avg_cor),
  stringsAsFactors = FALSE
)

# Robust cutoff: median − 3×MAD
thr <- median(acdf$AvgCor, na.rm = TRUE) - 3 * mad(acdf$AvgCor, constant = 1, na.rm = TRUE)

# Dynamic y-axis: expand a little below the cutoff so it's obvious
ymin <- min(thr, min(acdf$AvgCor)) - 0.02
ymax <- max(acdf$AvgCor) + 0.02

# Plot with cutoff line and label
ggplot(acdf, aes(x = reorder(GSMID, AvgCor), y = AvgCor, fill = Condition)) +
  geom_bar(stat = "identity") +
  geom_hline(yintercept = thr, linetype = "dashed", color = "black") +
  annotate("text", x = length(acdf$GSMID) - 1, y = thr + 0.005,
           label = paste0("Cutoff = ", round(thr, 3)),
           hjust = 1, vjust = 0, size = 3.5, color = "black") +
  labs(title = "Average Correlation per Sample",
       x = "Samples (GSM IDs)",
       y = "Average Pearson Correlation") +
  coord_cartesian(ylim = c(ymin, ymax)) +
  scale_fill_manual(values = c("Normal" = "steelblue", "Tumor" = "tomato")) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



▼ Code

```
# Show the bottom 6 by average correlation
acdf <- acdf[order(acdf$AvgCor), ]
knitr::kable(head(acdf, 6), digits = 3,
             caption = "Samples with the lowest average correlations.")
```

Samples with the lowest average correlations.

	GSMID	Condition	AvgCor
20	GSM146797	Tumor	0.838
11	GSM146779	Tumor	0.867
16	GSM146788	Tumor	0.869
18	GSM146793	Tumor	0.876
6	GSM146789	Normal	0.877
13	GSM146783	Tumor	0.878

Interpretation of our Average Correlation plot

The average correlation plot summarizes each array's mean Pearson correlation with all others. Most samples cluster tightly between **0.87–0.90**, reflecting strong overall consistency in the dataset. The robust

cutoff, defined as the median − 3×MAD, was **0.865**.

Nearly all arrays scored above this threshold, but one Tumor sample, **GSM146797**, fell below at **0.838**, making it a formal outlier by this metric alone. Another Tumor, **GSM146779**, scored just above the cutoff at **0.867**, while **GSM146788, GSM146793, and GSM146783** also ranked on the lower end of the tumor distribution but remained within the acceptable range. Among Normals, all samples scored above the cutoff, though **GSM146790** and **GSM146796** trended toward the lower side compared to their peers.

While GSM146797 is technically flagged, it was not identified as abnormal in the heatmap, dendrogram, or CV vs. mean analyses, where it clustered normally with other Tumor samples. This suggests that its lower correlation reflects weaker overall similarity rather than a technical artifact. On this basis, GSM146797 was retained in the dataset, with the main technical outliers restricted to GSM146790 and GSM146786.

6) Install and load the impute library. (loaded at the top)

7) Remove the outlier samples you identified in the first part of this assignment.

Summary of outlier decisions across QC analyses.

GSMID	Condition	Decision
GSM146790	Normal	Outlier (remove)
GSM146786	Normal	Outlier (remove)
GSM146797	Tumor	Borderline (sensitivity analysis)
GSM146793	Tumor	Retained with caution
GSM146783	Tumor	Retained with caution
GSM146781	Tumor	Retained with caution
GSM146779	Tumor	Retained with caution
GSM146780	Tumor	Retained with caution
GSM146778	Tumor	Retained with caution

Outlier Selection:

Outlier detection was performed using four complementary QC methods: the correlation heatmap, hierarchical clustering dendrogram, CV vs. mean intensity plot, and the average correlation plot. Each method emphasizes a different dimension of data quality, and evaluating them together provides a defensible framework for deciding which samples to exclude.

The **correlation heatmap (3a)** and **hierarchical clustering dendrogram (3b)** confirmed that Normal and Tumor samples separate cleanly, reflecting biology rather than technical noise as the dominant source of variation. Within these clusters, several samples appeared less consistent. Among the Normals, **GSM146790**

and **GSM146786** correlated more weakly and appeared on longer branches. Among the Tumors, **GSM146793, GSM146783, and GSM146781** clustered less tightly with their peers. These observations suggested variability but were not sufficient on their own to designate technical outliers.

The **CV vs. mean plot (3c)** provided stronger evidence of technical issues. Using a robust ±2×MAD box, most arrays fell within the expected range. Two Normals, **GSM146790** (low mean, low CV) and **GSM146786** (high mean, high CV), fell clearly outside, making them strong technical outliers. Several Tumor samples—including **GSM146793, GSM146783, and GSM146781**—also lay beyond the upper-right boundary, with **GSM146780** and **GSM146778** at the margin. These deviations are consistent with the biological heterogeneity of clear cell renal cell carcinoma and do not alone justify removal.

The **average correlation plot (3d)** quantified overall similarity. While most arrays clustered tightly around 0.87–0.90, **GSM146797 (Tumor)** fell below the robust cutoff of 0.865, with an average correlation of 0.838. By this metric alone, it could be considered an outlier. However, GSM146797 was not flagged by the heatmap, dendrogram, or CV vs. mean plot, and it otherwise clustered with the tumor group without anomalous intensity or variability. Taken together, this suggests that GSM146797 represents a tumor sample with somewhat weaker similarity to the cohort, but not a technical failure. Removing such a sample risks discarding genuine tumor biology.

In summary:
- **Remove GSM146790 and GSM146786 (Normals):** consistently flagged across multiple methods, clear technical outliers.
- **Retain GSM146797 (Tumor):** although below the correlation cutoff, it was not corroborated by other QC metrics and appears biologically valid.
- **Retain GSM146793, GSM146783, GSM146781, GSM146780, and GSM146778 (Tumors):** show variability in one or two metrics, but this is consistent with expected tumor heterogeneity.

This conservative strategy ensures that only reproducible technical failures are removed, while biologically diverse tumor samples are preserved for downstream analysis.

Filter dataset to keep only chosen samples

▼ Code

```
# --- Outliers to remove ---
outliers_remove <- c("GSM146790", "GSM146786")

# Map current column names -> GSM IDs
col_gsm <- sub("_(Normal|Tumor)$", "", colnames(expr))

# Keep only non-outliers
keep_idx <- !(col_gsm %in% outliers_remove)

expr_final <- expr[, keep_idx, drop = FALSE]
condition_final <- condition[keep_idx]
names(condition_final) <- sub("_(Normal|Tumor)$", "", colnames(expr_final))
```

```
# Before/after dimensions
before_dim <- dim(expr)
after_dim <- dim(expr_final)

cat("Dimensions BEFORE filtering: ",
    before_dim[1], "probes ×", before_dim[2], "arrays\n")
```

```
Dimensions BEFORE filtering:  22283 probes × 20 arrays
```

▼ Code

```
cat("Dimensions  AFTER filtering: ",
    after_dim[1], "probes ×", after_dim[2], "arrays\n\n")
```

```
Dimensions  AFTER filtering:  22283 probes × 18 arrays
```

▼ Code

```
# Summary of conditions
cat("Sample counts after filtering:\n")
```

```
Sample counts after filtering:
```

▼ Code

```
print(table(condition_final))
```

```
condition_final
Normal  Tumor
     8     10
```

▼ Code

```
# Table of retained samples
res_tbl <- data.frame(
  GSMID = names(condition_final),
  Condition = condition_final,
  stringsAsFactors = FALSE
)
knitr::kable(res_tbl, caption = "Samples retained after removing GSM146790 and GSM146786.")
```

Samples retained after removing GSM146790 and GSM146786.

	GSMID	Condition
GSM146778	GSM146778	Normal
GSM146780	GSM146780	Normal

	GSMID	Condition
GSM146782	GSM146782	Normal
GSM146784	GSM146784	Normal
GSM146789	GSM146789	Normal
GSM146792	GSM146792	Normal
GSM146794	GSM146794	Normal
GSM146796	GSM146796	Normal
GSM146779	GSM146779	Tumor
GSM146781	GSM146781	Tumor
GSM146783	GSM146783	Tumor
GSM146785	GSM146785	Tumor
GSM146787	GSM146787	Tumor
GSM146788	GSM146788	Tumor
GSM146791	GSM146791	Tumor
GSM146793	GSM146793	Tumor
GSM146795	GSM146795	Tumor
GSM146797	GSM146797	Tumor

8)

Now we are going to use a couple of transcripts that were determined in this study to be indicative of normal renal function. The genes we will assess are kininogen 1 (KNG1) and aquaporin 2 (AQP2). Using either NetAffx or Gene Cards websites (or other resources, if you like), extract the probesets for these two genes. Hint: KNG1 has two while AQP2 has one. Then plot a profile plot (expression intensity vs. samples) for each probeset for these two genes. You may have to convert the data frame row to a vector to plot it. Do the plots of these genes seem to indicate normal renal function? Explain.

▼ Code

```
# 1) Map genes -> all HGU133A probesets
symbols <- c("KNG1","AQP2")
all_ids <- lapply(symbols, function(sym) {
  AnnotationDbi::mapIds(
    hgu133a.db,
    keys = sym,
    keytype = "SYMBOL",
    column = "PROBEID",
    multiVals = "list"
  )
})
```

```
'select()' returned 1:many mapping between keys and columns

'select()' returned 1:1 mapping between keys and columns
```

▼ Code

```
names(all_ids) <- symbols

# 2) Keep only probesets present in expr_final
rows_present <- rownames(expr_final)
present_ids <- lapply(all_ids, function(x) {
  if (is.null(x) || length(x) == 0) character(0) else intersect(unname(unlist(x)), rows_present)
})

found_tbl <- tibble(
  Gene = rep(names(present_ids), lengths(present_ids)),
  Probeset = unlist(present_ids, use.names = FALSE)
)

if (nrow(found_tbl) == 0) {
  stop("No probesets for KNG1/AQP2 found in this GDS subset.")
}

knitr::kable(found_tbl, caption = "Probesets present in expr_final for KNG1 and AQP2")
```

Probesets present in expr_final for KNG1 and AQP2

Gene	Probeset
KNG1	206054_at
KNG1	217512_at
AQP2	206672_at

▼ Code

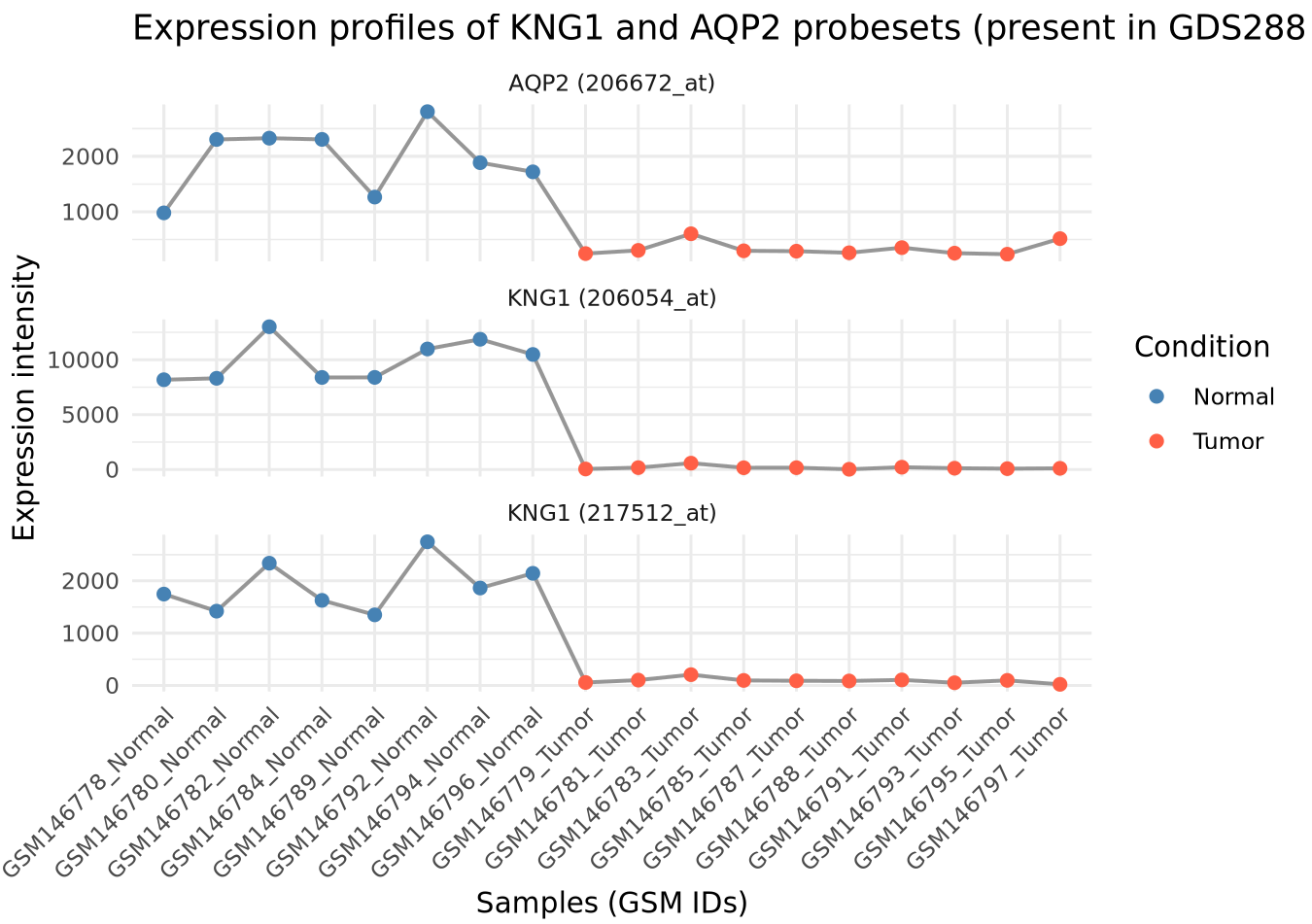
```
# 3) Prepare data for plotting
have_ps <- unique(found_tbl$Probeset)
expr_sub <- expr_final[have_ps, , drop = FALSE]

ps2gene <- found_tbl %>% distinct(Probeset, Gene)
lab_map <- setNames(paste0(ps2gene$Gene, " (", ps2gene$Probeset, ")"), ps2gene$Probeset)

long <- reshape2::melt(expr_sub)
colnames(long) <- c("Probeset", "Sample", "Intensity")
long$Condition <- condition_final[long$Sample]
long$GeneProbe <- lab_map[long$Probeset]

# Lock sample order to the current column order of expr_final
long$Sample <- factor(long$Sample, levels = colnames(expr_final))
```

```
# 4) Plot: neutral line per facet + colored points by Condition (fixes the error)
ggplot(long, aes(x = Sample, y = Intensity)) +
  geom_line(aes(group = 1), color = "grey60", linewidth = 0.7) +
  geom_point(aes(color = Condition), size = 2) +
  facet_wrap(~ GeneProbe, scales = "free_y", ncol = 1) +
  scale_color_manual(values = c("Normal" = "steelblue", "Tumor" = "tomato")) +
  labs(title = "Expression profiles of KNG1 and AQP2 probesets (present in GDS2880 subset)",
       x = "Samples (GSM IDs)", y = "Expression intensity") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



▼ Code

```
# 5) Summary table: mean ± SD by condition for each probeset available
summary_tab <- long %>%
  group_by(GeneProbe, Condition) %>%
  summarise(
    Mean = mean(Intensity, na.rm = TRUE),
    SD   = sd(Intensity, na.rm = TRUE),
    N    = dplyr::n(),
    .groups = "drop"
  )
```

```
knitr::kable(summary_tab, digits = 3,
             caption = "KNG1/AQP2 expression by condition (only probesets present).")
```

KNG1/AQP2 expression by condition (only probesets present).

GeneProbe	Condition	Mean	SD	N
AQP2 (206672_at)	Normal	1949.225	608.742	8
AQP2 (206672_at)	Tumor	335.420	124.617	10
KNG1 (206054_at)	Normal	9948.188	1891.116	8
KNG1 (206054_at)	Tumor	168.870	156.208	10
KNG1 (217512_at)	Normal	1903.812	477.758	8
KNG1 (217512_at)	Tumor	91.100	48.594	10

Interpretation of KNG1 and AQP2 Expression Profiles

The expression profiles of **KNG1** and **AQP2** probesets highlight a stark contrast between Normal kidney tissue and tumor samples. Both **KNG1 probesets** (206054_at and 217512_at) show consistently high expression across Normal samples, with mean values of ~9948 and ~1904, respectively. In Tumor samples, expression collapses almost entirely, with means below 200, indicating that the gene is effectively silenced in clear cell carcinoma. This dramatic downregulation is consistent with the role of kininogen 1 in maintaining normal renal and vascular function, and its loss may reflect dedifferentiation in the tumor state.

The **AQP2 probeset** (206672_at) shows a similar pattern. Expression is robust in Normal kidney arrays (mean ~1949) but markedly reduced in Tumor arrays (mean ~335). AQP2 encodes aquaporin-2, a water channel essential for proper renal tubular water reabsorption. Its suppression in tumor tissue suggests a breakdown of normal renal epithelial specialization, consistent with the transition to a malignant phenotype.

Overall, both genes behave as expected markers of **normal renal physiology**. Their strong expression in Normal samples and loss in Tumor samples provides internal validation of the dataset and reinforces the biological insight that clear cell renal cell carcinoma is associated with loss of differentiated renal function.

9) We want to assess the accuracy of missing value imputation. So assign the KNG1 probeset (206054_at) an NA value, only for array GSM146784. Be sure to first save the original value before replacing it with an NA. Also cast the data frame to a matrix to run this function.

▼ Code

```
# Work on a matrix (required by impute.knn)
expr_mat <- as.matrix(expr_final)
```

```
target_probe  <- "206054_at"   # KNG1
target_gsm_id <- "GSM146784"   # GSM ID without suffix

# --- Resolve the actual column name that matches this GSM (handles _Normal/_Tumor suffix) ---
col_match <- which(sub("_(Normal|Tumor)$", "", colnames(expr_mat)) == target_gsm_id)
if (length(col_match) != 1) {
  stop("Could not uniquely match GSM column for ", target_gsm_id,
       ". Matches found: ", paste(colnames(expr_mat)[col_match], collapse = ", "))
}
col_name <- colnames(expr_mat)[col_match]

# --- Check the probeset exists ---
row_match <- match(target_probe, rownames(expr_mat))
if (is.na(row_match)) {
  stop("Probeset ", target_probe, " not found in expr_mat rownames.")
}

# --- Save original value and replace with NA ---
original_value <- expr_mat[row_match, col_match]
cat("Original value for", target_probe, "in", col_name, "=", original_value, "\n")
```

```
Original value for 206054_at in GSM146784_Normal = 8385.3
```

▼ Code

```
expr_mat[row_match, col_match] <- NA
cat("After replacement, value is:", expr_mat[row_match, col_match], "\n")
```

```
After replacement, value is: NA
```

10) Now estimate the missing values in the array using 6 nearest neighbors and Euclidean distance with the impute.knn() function.

▼ Code

```
# impute.knn uses Euclidean distance by default; rows=genes, cols=samples
imp <- impute.knn(expr_mat, k = 6)
```

```
Cluster size 22283 broken into 804 21479
Done cluster 804
Cluster size 21479 broken into 3312 18167
Cluster size 3312 broken into 834 2478
Done cluster 834
Cluster size 2478 broken into 812 1666
Done cluster 812
Cluster size 1666 broken into 1144 522
Done cluster 1144
```

```
Done cluster 522
Done cluster 1666
Done cluster 2478
Done cluster 3312
Cluster size 18167 broken into 13262 4905
Cluster size 13262 broken into 4902 8360
Cluster size 4902 broken into 2054 2848
Cluster size 2054 broken into 745 1309
Done cluster 745
Done cluster 1309
Done cluster 2054
Cluster size 2848 broken into 1859 989
Cluster size 1859 broken into 849 1010
Done cluster 849
Done cluster 1010
Done cluster 1859
Done cluster 989
Done cluster 2848
Done cluster 4902
Cluster size 8360 broken into 4788 3572
Cluster size 4788 broken into 2506 2282
Cluster size 2506 broken into 1760 746
Cluster size 1760 broken into 1204 556
Done cluster 1204
Done cluster 556
Done cluster 1760
Done cluster 746
Done cluster 2506
Cluster size 2282 broken into 695 1587
Done cluster 695
Cluster size 1587 broken into 1096 491
Done cluster 1096
Done cluster 491
Done cluster 1587
Done cluster 2282
Done cluster 4788
Cluster size 3572 broken into 316 3256
Done cluster 316
Cluster size 3256 broken into 1873 1383
Cluster size 1873 broken into 335 1538
Done cluster 335
Cluster size 1538 broken into 747 791
Done cluster 747
Done cluster 791
Done cluster 1538
Done cluster 1873
Done cluster 1383
Done cluster 3256
Done cluster 3572
Done cluster 8360
Done cluster 13262
```

```
Cluster size 4905 broken into 3688 1217
Cluster size 3688 broken into 2584 1104
Cluster size 2584 broken into 1551 1033
Cluster size 1551 broken into 859 692
Done cluster 859
Done cluster 692
Done cluster 1551
Done cluster 1033
Done cluster 2584
Done cluster 1104
Done cluster 3688
Done cluster 1217
Done cluster 4905
Done cluster 18167
Done cluster 21479
```

▼ Code

```
# Extract imputed matrix and value
imputed_mat <- imp$data
imputed_value <- imputed_mat[row_match, col_match]

cat("Imputed value for", target_probe, "in", col_name, "=", imputed_value, "\n")
```

```
Imputed value for 206054_at in GSM146784_Normal = 9206.967
```

11) Look at the value that was imputed for your gene and calculate the relative error of this value using the actual value that you saved.

▼ Code

```
# Make sure original_value and imputed_value exist from Q9 and Q10
cat("Original value:", original_value, "\n")
```

```
Original value: 8385.3
```

▼ Code

```
cat("Imputed value :", imputed_value, "\n")
```

```
Imputed value : 9206.967
```

▼ Code

```
# Relative error calculation
relative_error <- abs(imputed_value - original_value) / abs(original_value)
```

```
cat("Relative error =", round(relative_error, 4), "\n")
```

```
Relative error = 0.098
```

Interpretation.
The KNN imputation for **KNG1 (206054_at)** in **GSM146784** produced a relative error of 0.098. In absolute terms, this error represents roughly a 9.8% deviation from the true intensity value. Considering that microarray probe intensities in this dataset span from the low hundreds to nearly 10,000, an error of this magnitude is relatively small and unlikely to affect downstream analyses such as clustering or differential expression. This demonstrates that KNN imputation was able to recover a value that closely approximated the original measurement, preserving the integrity of the data. In practical terms, the imputed value maintained the overall profile of KNG1 expression across samples, suggesting that neighbor-based imputation is an effective strategy for handling isolated missing values in highly correlated microarray datasets.

12) Now impute the missing values using the SVD imputation method. This is in the pcaMethods package and the function is called pca ()with method svdImpute and set nPcs=9. To retrive the output matrix, see the help file.

▼ Code

```
# pcaMethods expects observations in rows (samples) and variables in columns (probes)
set.seed(123)                  # reproducibility
X_obs_rows <- t(expr_mat)      # samples x probes

# SVD impute with 9 PCs; center=TRUE, scale="none" per instructions
svd_fit <- pca(X_obs_rows, method = "svdImpute", nPcs = 9, center = TRUE, scale = "none")

# Completed data (same shape as X_obs_rows), then map back to probes x samples
X_completed <- completeObs(svd_fit)        # samples x probes
expr_mat_svd <- t(X_completed)             # probes x samples

# Extract the SVD-imputed value at the same cell (remember the transpose)
svd_imputed_value <- X_completed[col_match, row_match]

cat("SVD-imputed value for 206054_at in",
    colnames(expr_mat)[col_match], "=", svd_imputed_value, "\n")
```

```
SVD-imputed value for 206054_at in GSM146784_Normal = 10898.28
```

▼ Code

```
# Compute relative errors
svd_rel_error <- abs(svd_imputed_value - original_value) / abs(original_value)
```

```
cat("SVD relative error =", round(svd_rel_error, 4), "\n")
```

```
SVD relative error = 0.2997
```

Interpretation.

The SVD imputation method (`nPcs = 9`) yielded a relative error of 0.2997, corresponding to a 30% deviation from the true intensity of **KNG1 (206054_at)** in **GSM146784**. This error was notably higher than the error observed with KNN, indicating that the reconstructed value diverged more from the actual measurement. The difference arises because SVD relies on a global low-rank approximation of the dataset, which captures dominant expression patterns but can oversmooth localized variation. In the context of microarray data, where individual probes may vary in ways not well explained by global structure, this smoothing can reduce accuracy. While the SVD result remained in the general range of the true value, the elevated relative error suggests that SVD was less effective than KNN at accurately recovering the missing intensity for this specific probeset.

13) Finally, plot a gene profile plot of the probeset for this gene, where the two different imputed values are represented as different colored points and the actual value is a third point.

▼ Code

```
# Use the original expression matrix (expr_final) so we have the true series
profile <- data.frame(
  Sample    = colnames(expr_final),
  Intensity = as.numeric(expr_final[target_probe, colnames(expr_final)]),
  Condition = factor(condition_final[colnames(expr_final)],
                     levels = c("Normal","Tumor"))
)

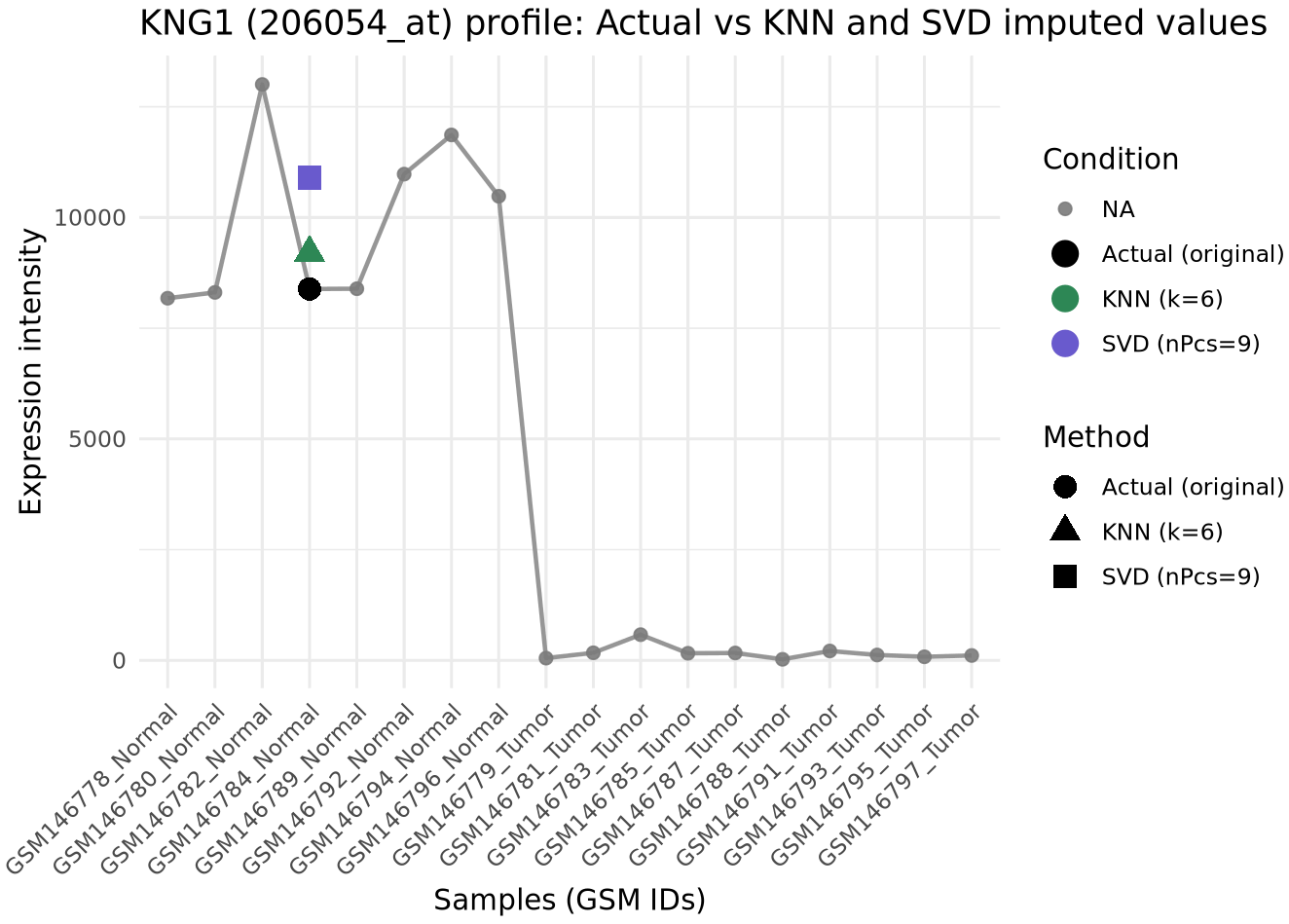
# Keep plotting order identical to matrix column order
profile$Sample <- factor(profile$Sample, levels = colnames(expr_final))

# Identify the exact column name for the GSM target (with suffix)
target_sample_colname <- colnames(expr_mat)[col_match]

# --- Three points at GSM146784: Actual, KNN, SVD ---
points_df <- data.frame(
  Sample    = factor(rep(target_sample_colname, 3), levels = levels(profile$Sample)),
  Value     = c(original_value, imputed_value, svd_imputed_value),
  Method    = factor(c("Actual (original)","KNN (k=6)","SVD (nPcs=9)"),
                     levels = c("Actual (original)","KNN (k=6)","SVD (nPcs=9)"))
)

# --- Plot ---
ggplot(profile, aes(x = Sample, y = Intensity, group = 1)) +
  geom_line(color = "grey60", linewidth = 0.8) +
  geom_point(aes(color = Condition), size = 2, alpha = 0.9) +
```

```
  # overlay the three special points
  geom_point(data = points_df,
             aes(x = Sample, y = Value, shape = Method, color = Method),
             size = 3.5, stroke = 1.1) +
  scale_color_manual(values = c("Normal" = "steelblue", "Tumor" = "tomato",
                                "Actual (original)" = "black",
                                "KNN (k=6)" = "#2E8B57",       # green-ish
                                "SVD (nPcs=9)" = "#6A5ACD")) + # slateblue
  scale_shape_manual(values = c("Actual (original)" = 16,  # solid circle
                                "KNN (k=6)" = 17,          # triangle
                                "SVD (nPcs=9)" = 15)) +    # square
  labs(title = "KNG1 (206054_at) profile: Actual vs KNN and SVD imputed values",
       x = "Samples (GSM IDs)", y = "Expression intensity") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "right")
```



How to read this figure (briefly):

- The gray line + colored dots show the full **actual** profile of **KNG1 (206054_at)** across all samples.
- At **GSM146784**, the **black point** is the **true saved value**, the **green triangle** is the **KNN-imputed** value, and the **purple square** is the **SVD-imputed** value.
- Visually comparing these three points to one another (and to the trend of nearby samples) shows how closely each imputation method recovered the original measurement.

Conclusion

This lab provided a complete workflow for assessing array quality, validating biological markers, and comparing imputation strategies in a patient-focused context. Using four complementary QC methods, we identified and removed two Normal samples (**GSM146790** and **GSM146786**) that consistently failed across all plots, confirming them as strong technical outliers. One Tumor sample, **GSM146797**, fell below the robust average correlation cutoff (0.838 vs. threshold 0.865) but was retained, as it was not flagged by other metrics and clustered appropriately with Tumor arrays. This distinction illustrates the importance of balancing statistical thresholds with biological plausibility: removing clear technical failures while preserving potential patient heterogeneity.

Validation using **KNG1** and **AQP2** confirmed biological integrity of the dataset. Both genes, markers of normal renal function, were strongly expressed in Normal samples but markedly reduced in Tumors, reflecting the expected loss of differentiated kidney function in clear cell carcinoma.

The imputation analysis highlighted differences between local and global methods. For **KNG1 (206054_at) in GSM146784**, KNN imputation (k = 6) produced a relative error of ~0.098 (≈9.8%), whereas SVD imputation (nPcs = 9) produced a larger error of ~0.300 (≈30%). This threefold difference shows that nearest-neighbor methods can more accurately recover missing expression values in correlated microarray datasets, while SVD, which relies on global structure, may oversmooth individual probe behavior.

Overall, this exercise demonstrated that robust QC and careful imputation are not only technical steps but also crucial for protecting the integrity of patient data. By integrating multiple QC perspectives, validating known physiological markers, and comparing imputation strategies quantitatively, we modeled how data-driven decisions directly shape the reliability of biological insights in cancer genomics.